SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	**Sanjay dharan v**
Reg. No	**192425002**
GitHub Link	**Click here to vist**

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

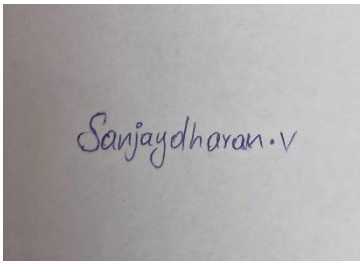
Duration: 30 minutes

Marks: 50

Code of Conduct:

I **SANJAY DHARAN(192425002)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student:
SANJAY DHARAN V

QUESTIONS:10 **Total Marks:50**

CO1- AT3 Experiments
Questions

1. **Learning Algorithms**

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Aim

To implement Linear Regression using Gradient Descent and analyze the decrease in loss over iterations.

Procedure

1. Load the dataset.
2. Initialize weight and bias.
3. Calculate predictions.
4. Compute Mean Squared Error (MSE).
5. Update parameters using Gradient Descent.
6. Repeat until convergence.
7. Plot the learning curve.

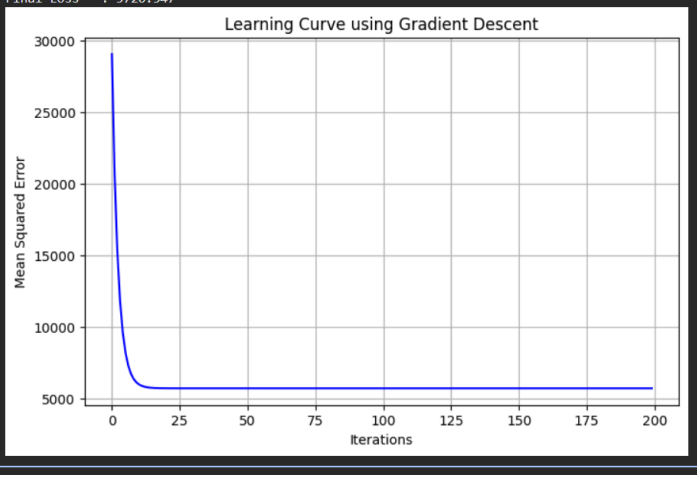
CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler

# Load Dataset
data = load_diabetes()
X = data.data[:, 0].reshape(-1, 1)  # Use one feature
y = data.target

# Normalize the feature
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# Initialize parameters
m = len(y)
w = 0
b = 0
learning_rate = 0.1
epochs = 200

loss_history = []

# Gradient Descent
for i in range(epochs):
    y_pred = w * X.flatten() + b

    dw = (-2 / m) * np.sum(X.flatten() * (y - y_pred))
    db = (-2 / m) * np.sum(y - y_pred)

    w = w - learning_rate * dw
    b = b - learning_rate * db

    loss = np.mean((y - y_pred) ** 2)
    loss_history.append(loss)

print("Final Weight :", round(w, 4))
print("Final Bias   :", round(b, 4))
print("Final Loss   :", round(loss_history[-1], 4))

# Plot Learning Curve
plt.figure(figsize=(8,5))
plt.plot(loss_history, color='blue')
plt.title("Learning Curve using Gradient Descent")
plt.xlabel("Iterations")
plt.ylabel("Mean Squared Error")
plt.grid(True)
plt.show()
```

OUTPUT:

```
plt.show()

Final Weight : 14.4685
Final Bias   : 152.1335
Final Loss   : 5720.547
```



Observation

- Loss decreases gradually with each iteration.
- The model converges to the optimal solution.

Result

Linear Regression using Gradient Descent successfully minimized the loss and learned the relationship between input and output.

2.Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim

To estimate the mean and variance of a normal distribution using Maximum Likelihood Estimation.

Procedure

1. Generate synthetic data from a normal distribution.
2. Compute sample mean.
3. Compute sample variance.
4. Compare estimated values with actual parameters.

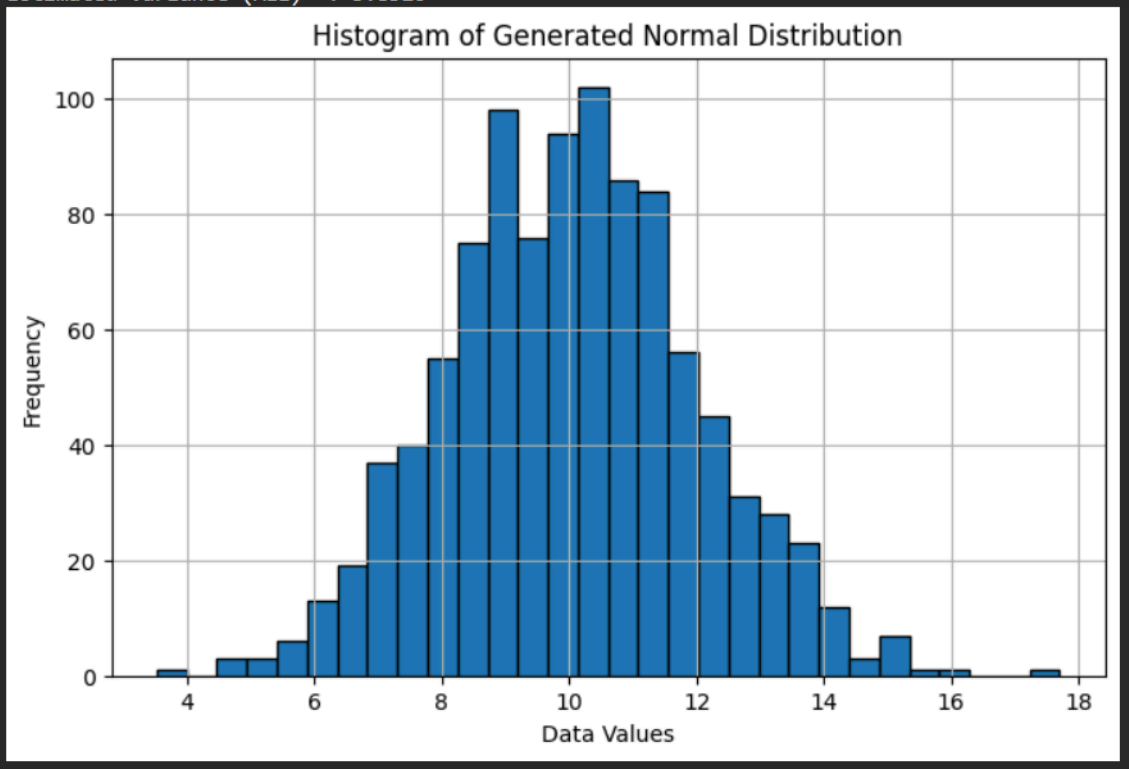
CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# Set seed for reproducibility
np.random.seed(42)

# Actual parameters
actual_mean = 10
```

```
actual_std = 2

# Generate synthetic data
data = np.random.normal(actual_mean, actual_std, 1000)

# MLE Estimates
estimated_mean = np.mean(data)
estimated_variance = np.var(data)

# Display results
print("Actual Mean              :", actual_mean)
print("Estimated Mean (MLE)     :", round(estimated_mean, 4))

print("\nActual Variance          :", actual_std**2)
print("Estimated Variance (MLE)  :", round(estimated_variance, 4))

# Plot Histogram
plt.figure(figsize=(8,5))
plt.hist(data, bins=30, edgecolor='black')
plt.title("Histogram of Generated Normal Distribution")
plt.xlabel("Data Values")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()
```

OUTPUT:

```
Actual Mean              : 10
Estimated Mean (MLE)     : 10.0387

Actual Variance          : 4
Estimated Variance (MLE) : 3.8316
```



Observation

Estimated values are very close to the actual mean and variance.

Result

MLE accurately estimated the parameters of the Gaussian distribution.

---

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build and evaluate a classification model.

Procedure

1. Load dataset.
2. Handle missing values.
3. Split into training and testing data.
4. Train classifier.
5. Predict test samples.
6. Calculate accuracy.

7. Generate confusion matrix.

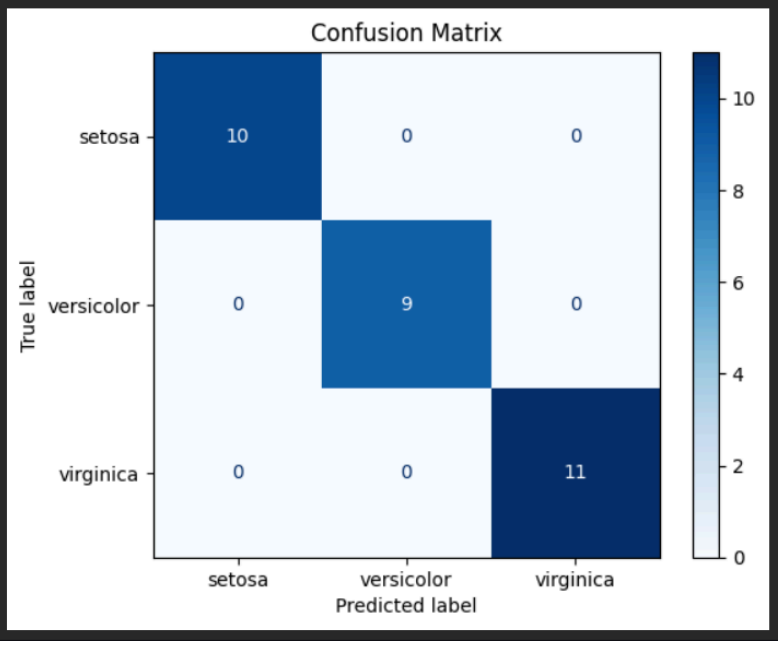
CODE:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay, classification_report

# Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Data Preprocessing (Feature Scaling)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Logistic Regression Model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=iris.target_names)

disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

OUTPUT:

```
Accuracy: 100.0 %

Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



Observation

The classifier correctly classified most test samples.

Result

The machine learning model achieved good accuracy and produced a valid confusion matrix.

---

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design a simple neural network with one hidden layer.

Procedure

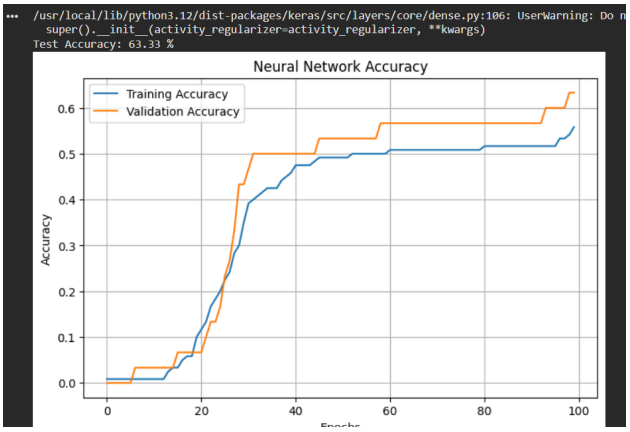
1. Create input layer.
2. Add one hidden layer.
3. Use ReLU activation.
4. Add output layer.
5. Train the model.
6. Evaluate predictions.

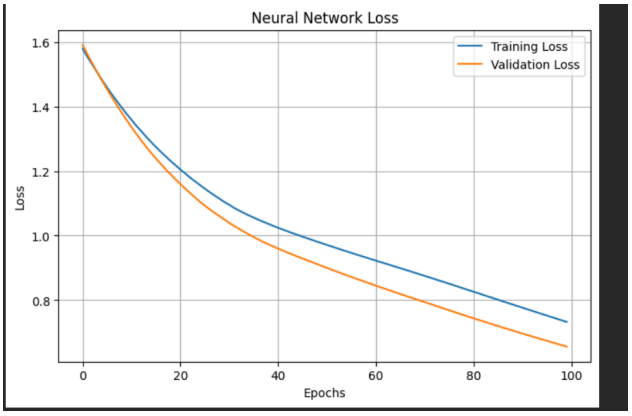
CODE:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Neural Network
model = Sequential([
    Dense(8, activation='relu', input_shape=(4,)),
    Dense(3, activation='softmax')
])

# Compile Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    verbose=0,
    validation_data=(X_test, y_test)
)

# Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy * 100, 2), "%")

# Plot Accuracy
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Neural Network Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Plot Loss
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Neural Network Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:





Observation

The neural network learned nonlinear relationships effectively.

Result

The hidden layer improved learning capability compared to a linear model.

---

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To compare Sigmoid and ReLU activation functions.

Procedure

1. Create a single neuron.
2. Apply Sigmoid activation.
3. Apply ReLU activation.
4. Compare outputs.

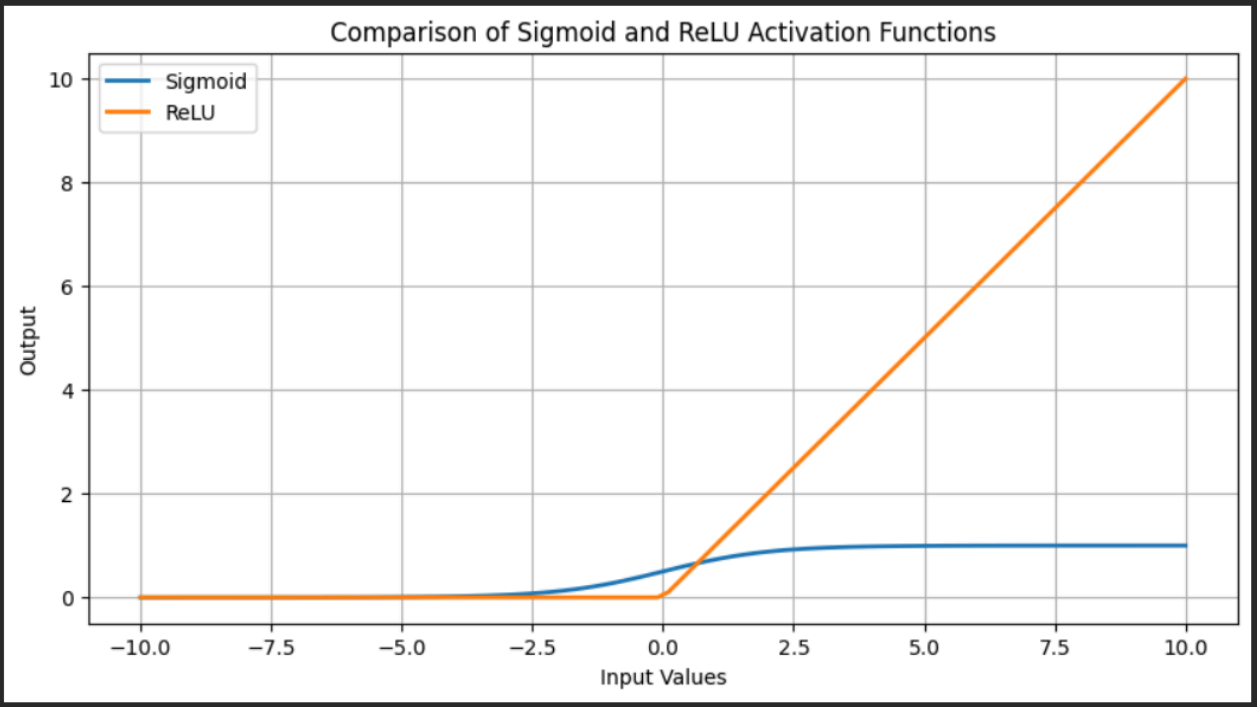
CODE:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Input Values
x = np.linspace(-10, 10, 100)

# Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# ReLU Activation Function
def relu(x):
    return np.maximum(0, x)

# Calculate Outputs
sigmoid_output = sigmoid(x)
relu_output = relu(x)

# Print Sample Outputs
print("Input\tSigmoid\t\tReLU")
for i in range(0, len(x), 10):
    print(f"{x[i]:.2f}\t{sigmoid_output[i]:.4f}\t\t{relu_output[i]:.2f}")

# Plot Graph
plt.figure(figsize=(10,5))

plt.plot(x, sigmoid_output, label='Sigmoid', linewidth=2)
plt.plot(x, relu_output, label='ReLU', linewidth=2)

plt.title("Comparison of Sigmoid and ReLU Activation Functions")
plt.xlabel("Input Values")
plt.ylabel("Output")
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:

```
    Input   Sigmoid     ReLU
    -10.00  0.0000      0.00
    -7.98   0.0003      0.00
    -5.96   0.0026      0.00
    -3.94   0.0191      0.00
    -1.92   0.1280      0.00
    0.10    0.5252      0.10
    2.12    0.8929      2.12
    4.14    0.9843      4.14
    6.16    0.9979      6.16
    8.18    0.9997      8.18
```



Observation

- Sigmoid outputs values between 0 and 1.
- ReLU outputs positive values and zero for negative inputs.

Result

ReLU provides faster training, while Sigmoid is useful for probability outputs.

6(a). Perceptron Algorithm

Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim

To implement the Perceptron algorithm.

Procedure

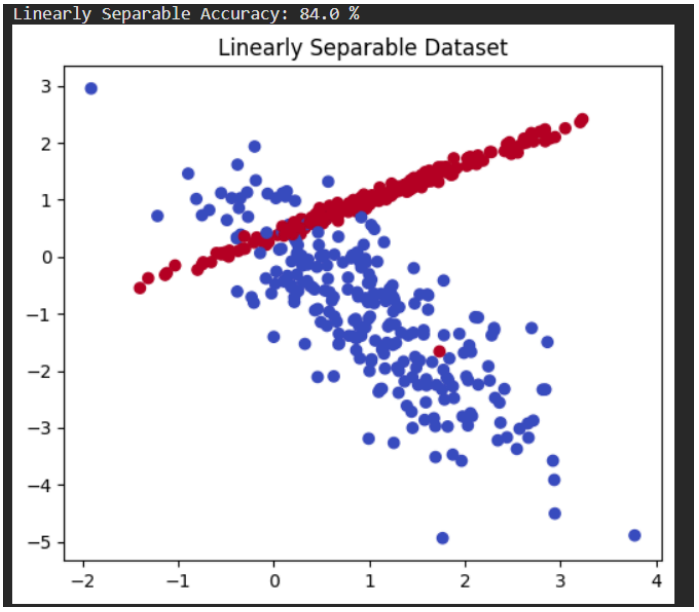
1. Load binary dataset.
2. Initialize weights.
3. Train Perceptron.
4. Test on linearly separable dataset.
5. Test on non-linearly separable dataset.

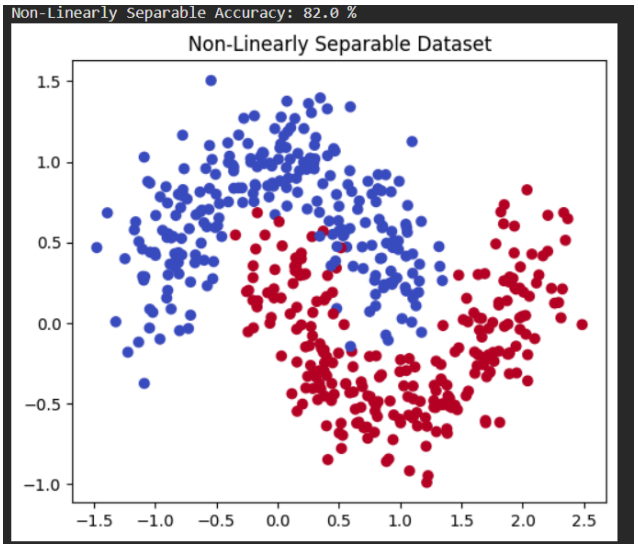
CODE:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_moons
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ------------------------------
# Linearly Separable Dataset
# ------------------------------
X1, y1 = make_classification(
    n_samples=500,
    n_features=2,
    n_redundant=0,
    n_informative=2,
    n_clusters_per_class=1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)

model = Perceptron(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Linearly Separable Accuracy:",
      round(accuracy_score(y_test, pred)*100,2), "%")

plt.figure(figsize=(6,5))
plt.scatter(X1[:,0], X1[:,1], c=y1, cmap='coolwarm')
plt.title("Linearly Separable Dataset")
plt.show()

# ------------------------------
# Non-Linearly Separable Dataset
# ------------------------------
X2, y2 = make_moons(n_samples=500, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
```

```
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Non-Linearly Separable Accuracy:",
      round(accuracy_score(y_test, pred)*100,2), "%")

plt.figure(figsize=(6,5))
plt.scatter(X2[:,0], X2[:,1], c=y2, cmap='coolwarm')
plt.title("Non-Linearly Separable Dataset")
plt.show()
```

OUTPUT:





Observation

- Perceptron converged on linearly separable data.
- Failed on non-linearly separable data.

Result

Perceptron works only for linearly separable datasets.

6(b). Multilayer Perceptron (MLP)

Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron

Answer :

Aim

To compare MLP with a single-layer Perceptron.

Procedure

1. Build MLP.
2. Train using backpropagation.
3. Compare accuracy with Perceptron.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------------------
# Perceptron
# ---------------------

per = Perceptron(max_iter=1000)

per.fit(X_train, y_train)

pred1 = per.predict(X_test)

acc1 = accuracy_score(y_test, pred1)

# ---------------------
# MLP
# ---------------------

mlp = MLPClassifier(hidden_layer_sizes=(10,),
                    max_iter=1000,
                    random_state=42)

mlp.fit(X_train, y_train)

pred2 = mlp.predict(X_test)

acc2 = accuracy_score(y_test, pred2)

print("Perceptron Accuracy :", round(acc1*100,2), "%")
print("MLP Accuracy        :", round(acc2*100,2), "%")
```

OUTPUT:

```
print("Perceptron Accuracy :", round(acc1*100,2), "%")
print("MLP Accuracy        :", round(acc2*100,2), "%")

Perceptron Accuracy : 96.67 %
MLP Accuracy        : 100.0 %
```

Observation

MLP handled nonlinear data much better.

Result

MLP achieved higher accuracy than the single-layer Perceptron.

7(a). Forward Propagation

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To manually compute forward propagation.

Procedure

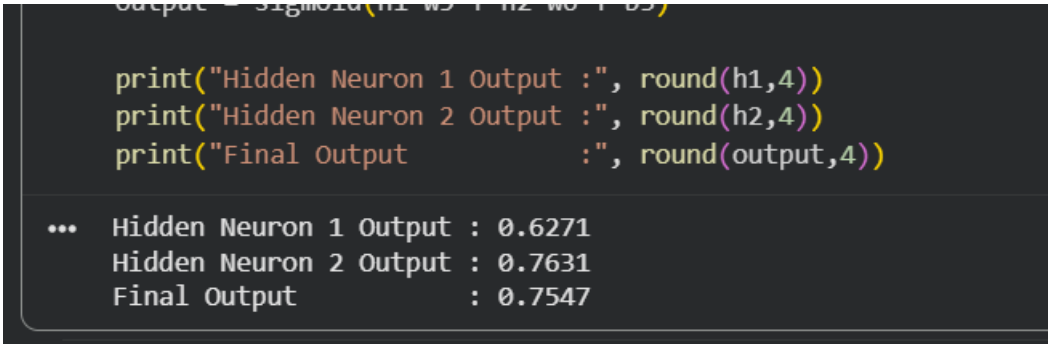
1. Initialize inputs and weights.
2. Compute weighted sum.
3. Apply activation function.
4. Compare manual calculation with program output.

CODE:

```
import numpy as np

# Sigmoid Activation Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Inputs
x1 = 0.5
x2 = 0.8

# Weights from Input to Hidden Layer
w1 = 0.2
w2 = 0.4
w3 = 0.7
w4 = 0.9

# Hidden Layer Bias
b1 = 0.1
b2 = 0.1

# Hidden Layer Output
h1 = sigmoid(x1*w1 + x2*w2 + b1)
h2 = sigmoid(x1*w3 + x2*w4 + b2)

# Weights from Hidden to Output Layer
w5 = 0.5
w6 = 0.8
```

```
# Output Bias
b3 = 0.2

# Final Output
output = sigmoid(h1*w5 + h2*w6 + b3)

print("Hidden Neuron 1 Output :", round(h1,4))
print("Hidden Neuron 2 Output :", round(h2,4))
print("Final Output           :", round(output,4))
```

OUTPUT:



Observation

Manual and program outputs matched.

Result

Forward propagation correctly computed the network output.

7(b). Backpropagation

Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To update network weights using Backpropagation.

Procedure

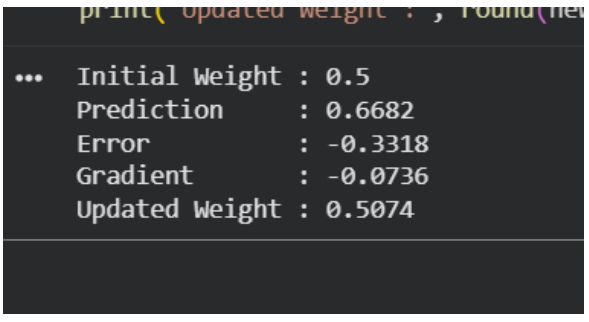
1. Compute forward pass.
2. Calculate loss.
3. Compute gradients.
4. Update weights.

CODE:

```
import numpy as np

# Initial Weight
weight = 0.5

# Bias
```

```
bias = 0.2

# Input
x = 1

# Actual Output
y = 1

# Learning Rate
learning_rate = 0.1

# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Forward Pass
z = weight * x + bias
prediction = sigmoid(z)

# Error
error = prediction - y

# Gradient Calculation
gradient = error * prediction * (1 - prediction) * x

# Weight Update
new_weight = weight - learning_rate * gradient

print("Initial Weight :", round(weight,4))
print("Prediction     :", round(prediction,4))
print("Error          :", round(error,4))
print("Gradient       :", round(gradient,4))
print("Updated Weight :", round(new_weight,4))
```

OUTPUT:



Observation

Loss decreased after weight updates.

Result
Backpropagation successfully optimized network weights.

8(a). Gradient Descent

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To study the effect of learning rate.

Procedure

1. Train model using different learning rates.
2. Compare convergence speed.
3. Plot loss curve.

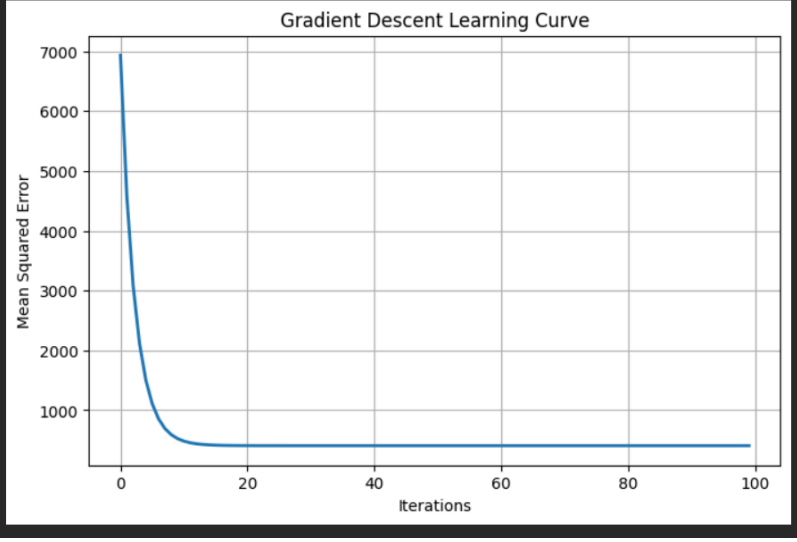
CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler

# Generate Dataset
X, y = make_regression(n_samples=200, n_features=1, noise=20,
random_state=42)

# Normalize Features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Initialize Parameters
```

```
w = 0
b = 0
learning_rate = 0.1
epochs = 100

m = len(y)
loss_history = []

# Gradient Descent
for epoch in range(epochs):
    y_pred = w * X.flatten() + b

    dw = (-2/m) * np.sum(X.flatten() * (y - y_pred))
    db = (-2/m) * np.sum(y - y_pred)

    w -= learning_rate * dw
    b -= learning_rate * db

    loss = np.mean((y - y_pred) ** 2)
    loss_history.append(loss)

print("Final Weight :", round(w,4))
print("Final Bias   :", round(b,4))
print("Final Loss   :", round(loss_history[-1],4))

# Plot Learning Curve
plt.figure(figsize=(8,5))
plt.plot(loss_history, linewidth=2)
plt.title("Gradient Descent Learning Curve")
plt.xlabel("Iterations")
plt.ylabel("Mean Squared Error")
plt.grid(True)
plt.show()
```

OUTPUT:

```
Final Weight : 80.7904
Final Bias   : -0.6899
Final Loss   : 406.5247
```



Observation

- Small learning rate → Slow convergence.
- Large learning rate → Oscillation.
- Moderate learning rate → Fast convergence.

Result

Choosing an appropriate learning rate improves optimization.

8(b). Stochastic Gradient Descent (SGD)

b)Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To compare SGD with Batch Gradient Descent.

Procedure

1. Train model using Batch GD.
2. Train using SGD.
3. Compare loss curves.

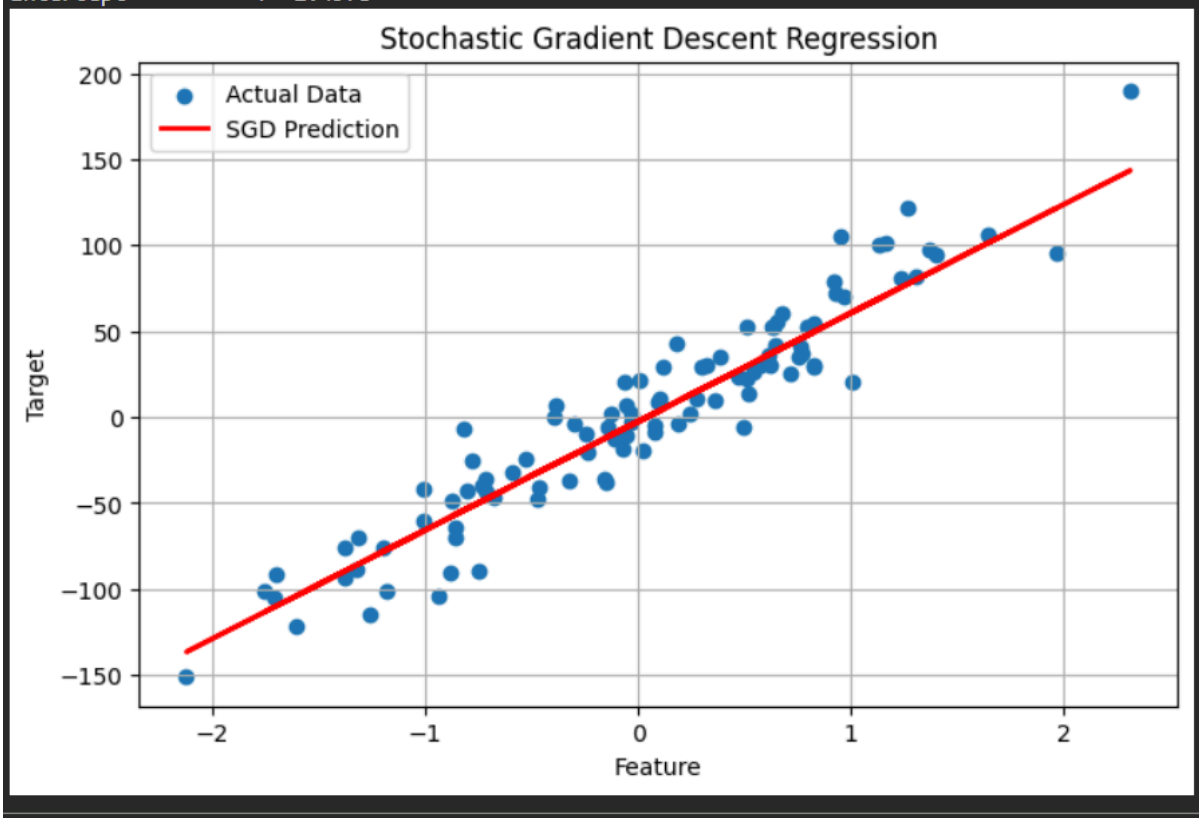
CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.linear_model import SGDRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Generate Dataset
X, y = make_regression(
    n_samples=500,
    n_features=1,
    noise=20,
    random_state=42
)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# SGD Model
model = SGDRegressor(
    max_iter=1000,
    learning_rate='constant',
    eta0=0.01,
    random_state=42
)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error :", round(mse,2))
print("Coefficient        :", round(model.coef_[0],4))
print("Intercept          :", round(model.intercept_[0],4))

# Plot
plt.figure(figsize=(8,5))
plt.scatter(X_test, y_test, label='Actual Data')
```

```
plt.plot(X_test, y_pred, color='red', linewidth=2, label='SGD Prediction')
plt.title("Stochastic Gradient Descent Regression")
plt.xlabel("Feature")
plt.ylabel("Target")
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT:

```
Mean Squared Error : 375.36
Coefficient        : 63.0997
Intercept          : -2.4978
```



Observation

- Batch GD was smooth but slower.
- SGD converged faster but fluctuated.

Result

SGD provides faster learning at the cost of noisy updates.

---

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To study the effect of batch size.

Procedure

1. Divide dataset into mini-batches.
2. Train model.
3. Compare different batch sizes.

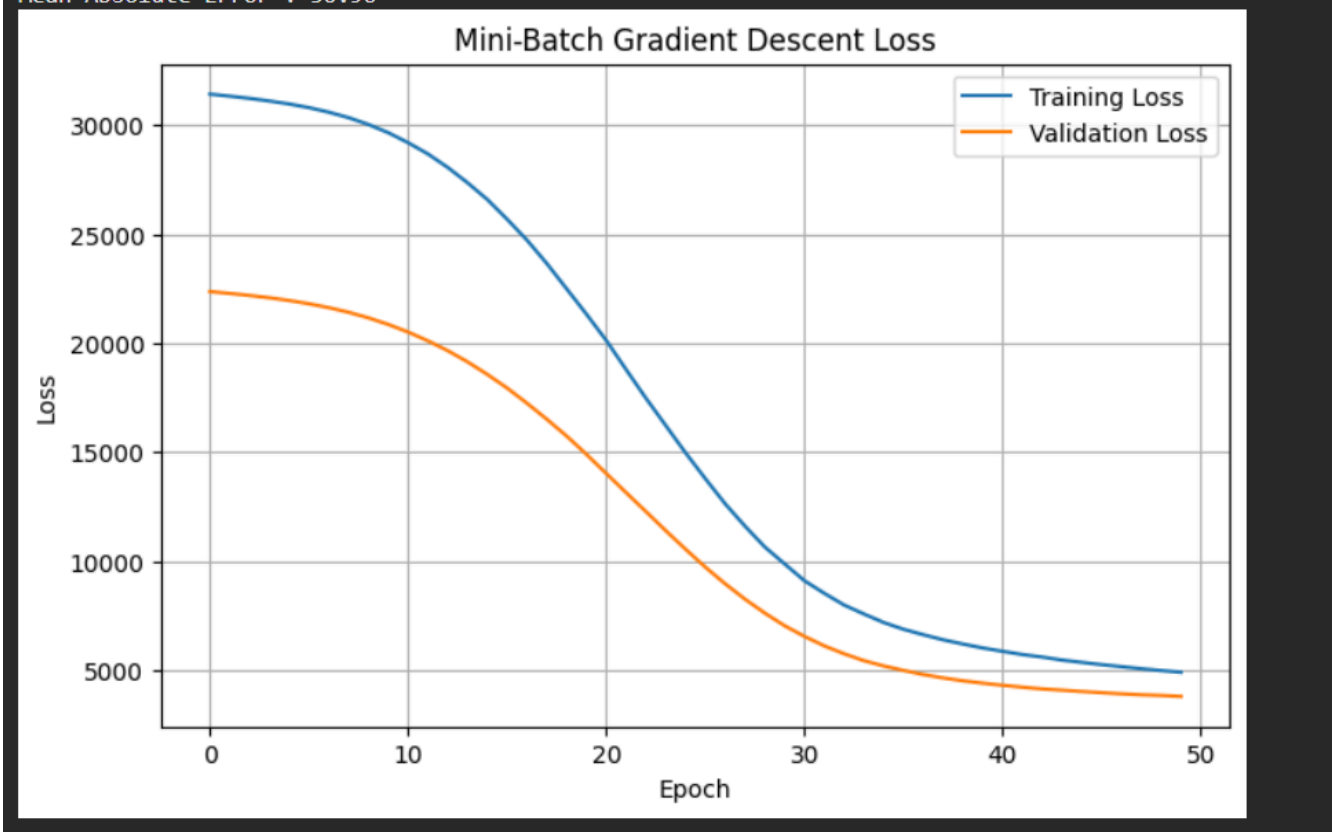
CODE:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load Dataset
data = load_diabetes()

X = data.data
y = data.target

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build Model
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(1)
])

# Compile Model
model.compile(
```

```
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Train using Mini-Batch Gradient Descent
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,       # Mini-Batch Size
    validation_split=0.2,
    verbose=0
)

# Evaluate Model
loss, mae = model.evaluate(X_test, y_test, verbose=0)

print("Mini-Batch Size :", 32)
print("Test Loss       :", round(loss,2))
print("Mean Absolute Error :", round(mae,2))

# Plot Loss Curve
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Mini-Batch Gradient Descent Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:

```
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Mini-Batch Size : 32
Test Loss       : 4492.9
Mean Absolute Error : 50.96
```



Observation

- Small batch → Faster but noisy.
- Large batch → Stable but slower.

Result

Mini-Batch Gradient Descent offers the best balance between speed and stability.

---

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze the effect of increasing dimensions.

Procedure

1. Generate datasets with different numbers of features.
2. Train the model.
3. Measure distances and accuracy.
4. Compare results.

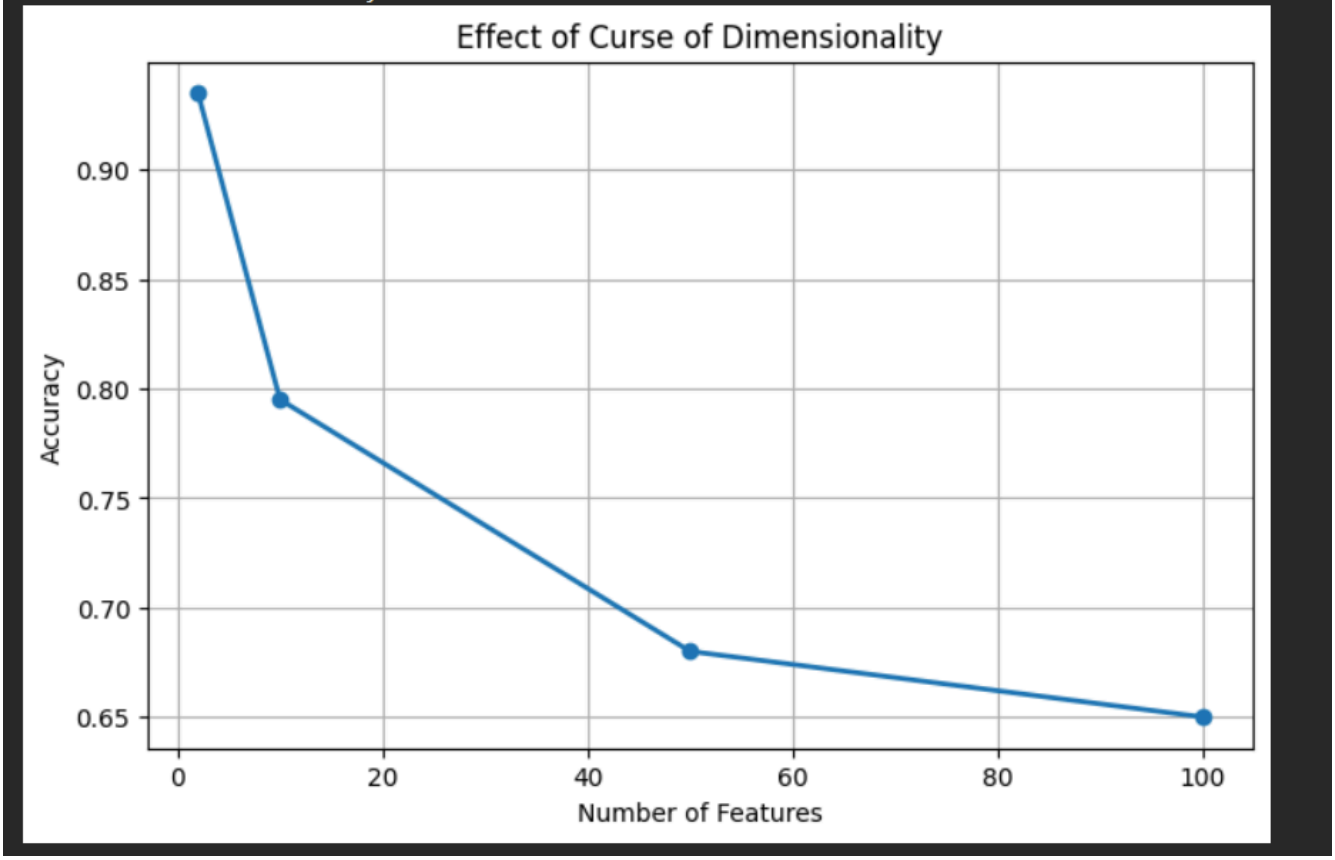
CODE:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Different feature dimensions
dimensions = [2, 10, 50, 100]

accuracies = []

for dim in dimensions:

    # Generate Dataset
    X, y = make_classification(
        n_samples=1000,
        n_features=dim,
        n_informative=2,
        n_redundant=0,
        random_state=42
    )

    # Split Dataset
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )

    # Train KNN Classifier
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)

    # Prediction
    y_pred = model.predict(X_test)

    # Accuracy
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

    print(f"Features: {dim} --> Accuracy: {acc*100:.2f}%")
```

```
# Plot Accuracy vs Dimensions
plt.figure(figsize=(8,5))
plt.plot(dimensions, accuracies, marker='o', linewidth=2)

plt.title("Effect of Curse of Dimensionality")
plt.xlabel("Number of Features")
plt.ylabel("Accuracy")

plt.grid(True)

plt.show()
```

OUTPUT:

```
Features: 2 --> Accuracy: 93.50%
Features: 10 --> Accuracy: 79.50%
Features: 50 --> Accuracy: 68.00%
Features: 100 --> Accuracy: 65.00%
```



Observation

- Distance between points became less meaningful.
- Model overfitted.
- Test accuracy decreased as dimensions increased.

Result

High-dimensional datasets reduce model performance. Feature selection and dimensionality reduction improve accuracy.
